

Secuestro de PATH

Objetivos de aprendizaje

Demostrar resolución por PATH, entorno heredado y directorio escribible antes de introducir un ejecutable marcador; distinguir cadenas presentes de llamadas alcanzables.

Prerrequisitos

PATH, execvp, shell, permisos de directorio, cron/sudo/SUID y credenciales efectivas.

Fundamentos técnicos

El vector requiere: consumidor privilegiado invoca un nombre sin ruta; usa búsqueda PATH; el PATH efectivo contiene antes un directorio escribible; y la rama se ejecuta. strings solo descubre texto. Exportar PATH en una shell no afecta a un proceso que fija su entorno. En SUID, shells e intérpretes pueden bajar privilegios.

Modelo mental

```
proceso privilegiado -> nombre relativo -> PATH efectivo -> primer ejecutable
                    -> permisos/identidad -> efecto
```

Superficie de ataque

Scripts root, tareas, wrappers, binarios que usan execvp/system, reglas sudo con entorno y servicios que llaman utilidades auxiliares.

Cómo identificarla

- Traza muestra búsqueda o ejecución de nombre relativo.
- PATH del proceso privilegiado se conoce.
- Un directorio anterior es escribible y atravesable.
- Un marcador se ejecuta con identidad esperada.

Preguntas que debo hacerme

1. ¿La llamada usa ruta absoluta?
2. ¿Qué PATH ve el consumidor?
3. ¿Cuál es el primer candidato ejecutable?
4. ¿Puedo escribir y atravesar ese directorio?
5. ¿El intérprete conserva privilegios?

Prueba mínima

Crear un ejecutable marcador con el nombre esperado que escriba id a /tmp, en un directorio de laboratorio ya incluido en el PATH privilegiado. No sustituir binarios reales ni lanzar shell.

Construcción progresiva del payload

1. Confirmar llamada alcanzable.
2. Capturar PATH efectivo.
3. Ordenar candidatos.
4. Verificar permisos de directorio.
5. Introducir marcador.
6. Ejecutar/esperar consumidor, confirmar y limpiar.

Anatomía de los payloads

- Contexto de entrada: cron root ejecuta wrapper que llama backup-helper.
- Sintaxis original: búsqueda por PATH.
- Entrada controlada: archivo /opt/team/bin/backup-helper.
- Transformaciones conocidas: resolución PATH izquierda a derecha.
- Parser final: kernel al ejecutar el candidato.
- Sink: proceso auxiliar como root.
- Primitiva: ejecutar marcador root.
- Payload mínimo: script que guarda id en /tmp/path-proof.
- Significado de cada componente: el nombre coincide con el auxiliar; el marcador registra la identidad sin persistencia.
- Resultado esperado: cron selecciona el candidato anterior al binario legítimo y crea la prueba.
- Control negativo: retirar marcador y observar helper legítimo.
- Restricción observada: exportar PATH del alumno no afecta cron.
- Por qué falla la variante básica: se modificó el entorno equivocado.
- Hipótesis de adaptación: usar un directorio ya presente en PATH de cron.
- Payload adaptado: marcador en /opt/team/bin, si es escribible y precede al legítimo.
- Por qué debería funcionar: modifica un componente que sí pertenece al entorno heredado por el consumidor privilegiado.
- Evidencia: archivo root y traza de la ruta ejecutada.
- Cuándo no funcionaría: PATH fijo sin directorio escribible, ruta absoluta, hash/cache o drop.

Variaciones según el contexto

Shell, execvp y bibliotecas buscan PATH; execve con ruta absoluta no. Sudo puede aplicar secure_path; systemd suele definir entorno explícito; un SUID puede sanearlo o perder EUID al invocar shell.

Filtros y bypasses

Si el PATH privilegiado no es controlable, no existe adaptación mediante export local. Buscar otro consumidor o cerrar la hipótesis.

Evidencias de confirmación

Traza de ruta ejecutada y marcador con identidad privilegiada. Una cadena cat en el binario no confirma llamada.

Escalado de impacto

Demostrar ejecución mínima, restaurar y documentar consumidor/entorno. No reemplazar utilidades del sistema.

Errores frecuentes

- Concluir desde strings.
- Exportar PATH solo en la sesión del alumno.
- Ignorar secure_path.
- Crear /bin/bash como payload y perder EUID.
- No comprobar orden ni permisos x del directorio.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
marcador no se ejecuta	PATH distinto o rama no alcanzada	entorno/traza del consumidor
ejecuta helper legítimo	directorio posterior/cache	orden PATH y type -a/traza
ejecuta marcador como alumno	consumidor no privilegiado	UID/PID correlacionado
sudo ignora PATH	secure_path	sudoers/entorno dentro
shell pierde EUID	modo no privilegiado	marcador directo sin shell y manual Bash

Mitigaciones

Rutas absolutas, PATH mínimo no escribible, entorno fijado, permisos de directorio, evitar shell, drop temprano y revisión de consumidores privilegiados.

Relación con pentesting y certificaciones

Se evalúa la demostración de resolución y entorno, no la creación de un binario llamado cat.

Caso guiado

Caso A — Básico: helper de backup

Cron root tiene PATH /opt/team/bin:/usr/bin; el primer directorio es escribible y el wrapper llama backup-helper. Marcador produce archivo root. Conclusión: PATH hijacking confirmado.

Caso de adaptación

Caso B — PATH exportado no se hereda

Anteponer /tmp en la sesión no cambia el cron. Experimento: capturar PATH del proceso. Resultado: cron lo fija. Conclusión: la variante falla por entorno; si ningún directorio de ese PATH es escribible, se descarta.

Caso C — Transferencia: servicio y execvp

Un servicio root llama compressor mediante execvp y define PATH con un directorio de plugins escribible. El alumno usa marcador, confirma ruta y limpia. Misma primitiva, consumidor distinto.

Caso D — Falso positivo: cadena no alcanzable

strings muestra service, pero la traza del flujo probado nunca ejecuta ese nombre; pertenece a una función de depuración deshabilitada. Conclusión: no hay sink alcanzado.

Ejercicios

1. Ordena candidatos de tres PATH distintos.
2. Diseña un marcador sin shell persistente.
3. Distingue entorno interactivo, sudo y cron.
4. Explica por qué strings es solo una pista.

Resumen

PATH hijacking exige nombre relativo, búsqueda efectiva, directorio escribible anterior y consumidor privilegiado alcanzable.

Chuleta operativa

1. Consumidor/UID.
2. Llamada relativa.
3. PATH efectivo.
4. Orden y permisos.
5. Marcador/control.
6. Drop/limpieza.

Referencias

- [environ\(7\)](#)
- [sudoers\(5\)](#)
- [Concepto interno](#) (path-hijacking)

Navegación

Anterior: [Cron](#). Índice: [curso](#). Práctica: [cuaderno](#).